ELOGEST RECEBE FÁCIL

# Manifesto de entrega

Revisão técnica, correções no frontend e construção do backend

**Data:** 23 de setembro de 2026

**Projeto:** Sistema de cobranças e contas a receber

**Escopo:** Frontend (HTML/CSS/JS) + novo backend (Python · FastAPI · SQLite)

**Situação:** Entregue e verificado — 16 testes automatizados passando

# 1. Resumo executivo

O projeto começou como um protótipo 100% frontend, sem servidor: clientes, débitos e usuários eram simulados no armazenamento do navegador (localStorage). A revisão técnica encontrou 14 problemas — 5 bugs que afetavam o uso diário, 4 riscos de segurança e 5 pontos de consistência.

Na sequência, foram entregues:

- **Correção dos 5 bugs críticos** no frontend (fuso horário, valor do carrinho, débito duplicado, falha de gravação silenciosa e perda de dados).
- **Um backend real** em Python + FastAPI + SQLite, com login seguro, validação no servidor, dinheiro em centavos e transações.
- **Integração sem reescrever telas:** o frontend passou a ter dois modos — API (dados no servidor) e demonstração (dados no navegador) — escolhidos automaticamente.
- **Verificação:** 16 testes automatizados da API e testes manuais no navegador nos dois modos.

| Indicador | Resultado |
|---|---|
| Problemas encontrados na revisão | 14 |
| Corrigidos ou resolvidos | 11 integralmente · 2 parcialmente |
| Pendentes (fora do escopo) | Itens menores listados na seção 9 |
| Endpoints de API criados | 17 |
| Testes automatizados | 16 de 16 passando |
| Arquivos criados / alterados | 24 novos · 23 alterados |

# 2. Ponto de partida

Estado do projeto antes do trabalho:

- Frontend em HTML, CSS e JavaScript puro, sem framework e sem etapa de build.
- Nenhum backend: o arquivo js/store.js simulava uma API gravando no localStorage.
- Login apenas simulado — qualquer pessoa podia “entrar” editando o armazenamento do navegador.
- Senhas, clientes e débitos visíveis e editáveis por quem tivesse acesso ao navegador.
- Arquitetura bem preparada para receber um backend: todas as telas já usavam funções assíncronas de um único ponto (window.ElogestStore).

# 3. Revisão técnica — achados e situação

A tabela abaixo resume cada achado da revisão, sua gravidade e o que foi feito.

| # | Achado | Gravidade | Situação |
|---|---|---|---|
| 1 | Data de “hoje” calculada em UTC: débitos que vencem hoje apareciam como vencidos após as 21h | Bug crítico | **Corrigido (frontend e backend)** |
| 2 | Valor do débito somava linhas do carrinho que depois eram descartadas; aceitava valores negativos | Bug crítico | **Corrigido** |
| 3 | Débito e itens gravados em duas etapas: falha parcial gerava débito duplicado | Bug crítico | **Corrigido (operação única)** |
| 4 | Erro de gravação era ignorado e a tela mostrava “sucesso” | Bug crítico | **Corrigido** |
| 5 | Atualizar os dados de exemplo apagava os dados cadastrados pelo usuário | Bug crítico | **Corrigido** |

| # | Achado | Gravidade | Situação |
|---|---|---|---|
| 6 | Login burlável editando o navegador; sessão sem expiração | Segurança | **Resolvido no backend (JWT de 8 h)** |
| 7 | Senhas guardadas em texto puro | Segurança | **Resolvido no backend (scrypt)** |
| 8 | Sem validação real: CPF sem dígito verificador, sem checar duplicidade, campos livres (id, status) | Segurança | **Corrigido (frontend e backend)** |
| 9 | Telefone e iniciais inseridos na página sem escape (risco de XSS) | Segurança | **Corrigido** |
| 10 | "Ticket médio" com duas fórmulas diferentes no perfil e no relatório de regiões | Consistência | **Documentado (não unificado)** |
| 11 | Sem registro de quando/quem deu baixa; sem editar ou excluir débito | Consistência | **Parcial: data e autor do pagamento gravados** |
| 12 | Regras de negócio espalhadas nas telas | Consistência | **Corrigido (movidas para store/API)** |
| 13 | Dinheiro somado em ponto flutuante | Consistência | **Corrigido (centavos inteiros)** |
| 14 | Menores: aviso some ao trocar de página; ID pulado; abas simultâneas; WhatsApp com 12 dígitos | Menor | **Parcial: ID e concorrência resolvidos** |

# 4. Correções no frontend

## 4.1 Os cinco bugs críticos

- **Fuso horário.** Datas passaram a ser montadas no horário local (novas funções dataISO/hojeISO em format.js e formatarISO em store.js).
- **Carrinho.** O total considera apenas itens com nome e quantidade — os mesmos que são gravados. Campos numéricos não aceitam valores negativos.
- **Débito atômico.** Débito e itens são enviados numa única chamada (Debitos.criar com itens); ou grava tudo, ou nada.
- **Falha de gravação.** A gravação lança erro quando o navegador não consegue salvar, e gravações múltiplas são desfeitas em caso de falha.
- **Dados de exemplo.** Só são criados quando a base está vazia. Uma migração corrige os CPFs/CNPJs antigos de exemplo sem tocar nos dados do usuário.

## 4.2 Melhorias adicionais

- Validação de CPF/CNPJ com dígitos verificadores e bloqueio de documento duplicado.
- Vencimento não pode ser anterior à data de criação (validado na tela e no servidor).
- Mensagens de erro do servidor exibidas diretamente nos avisos (ex.: "Já existe um cliente cadastrado com este CPF/CNPJ.").
- Escape de telefone e iniciais antes de inserir na página.
- Ranking "comprados este mês" e filtro de débitos por cliente movidos para o store.
- Limite de 500 caracteres na descrição do débito; rodapé indica quando o sistema está em modo demonstração.
- **Relatório de regiões (apontado por você durante a entrega):** o ranking é ordenado pelo valor vendido, mas mostrava só a quantidade — a Farinha (1 × R$ 210,00) aparecia acima da Massa (40 × R$ 174,00) sem explicação. Agora cada produto mostra quantidade e valor, e o painel se chama "Produtos que mais faturam".

# 5. Backend construído

## 5.1 Tecnologia

| Componente | Escolha | Motivo |
|---|---|---|
| Linguagem | Python 3.14 | Disponível na máquina (Node.js não estava instalado) |
| Framework | FastAPI 0.141 | Validação automática e documentação em /docs |
| Banco de dados | SQLite (modo WAL) | Zero instalação; suficiente para uma loja |
| Autenticação | JWT (PyJWT) + scrypt | Padrão de mercado; hash de senha sem dependência extra |
| Fuso horário | America/Sao_Paulo (tzdata) | Status de vencimento correto no Brasil |

## 5.2 Modelo de dados

| Tabela | Conteúdo | Destaques |
|---|---|---|
| usuarios | Login, nome, hash da senha | Usuário único (sem diferenciar maiúsculas) |
| clientes | Dados cadastrais e bairro | CPF/CNPJ único (pelos dígitos) |
| debitos | Valor, datas, descrição, status | Valor em centavos · pago_em · pago_por · criado_por |
| itens_compra | Produto, categoria, quantidade, valor unitário | Apagados junto com o débito ou o cliente |

## 5.3 Endpoints da API

*Todos exigem login (cabeçalho Authorization: Bearer <token>), exceto o próprio login.*

| Método | Rota | Função |
|---|---|---|
| POST | /api/auth/login | Entrar; devolve token e usuário |
| GET | /api/auth/me | Dados do usuário logado |
| GET | /api/clientes | Listar clientes |
| GET | /api/clientes/{id} | Buscar um cliente |
| POST | /api/clientes | Cadastrar cliente |
| PUT | /api/clientes/{id} | Atualizar cliente |
| DELETE | /api/clientes/{id} | Excluir cliente (e seus débitos e itens) |
| GET | /api/debitos | Listar débitos (filtros: cliente_id, status) |
| POST | /api/debitos | Registrar débito com itens, numa transação |
| PATCH | /api/debitos/{id}/pagar | Marcar como pago |
| GET | /api/debitos/{id}/itens | Itens de um débito |
| PUT | /api/debitos/{id}/itens | Substituir itens de um débito em aberto |
| GET | /api/itens | Itens comprados por um cliente |
| GET | /api/itens/ranking | Produtos mais comprados (por cliente e/ou mês) |
| GET | /api/itens/gasto-total | Total em itens detalhados de um cliente |
| GET | /api/regioes/resumo | Relatório por bairro |
| GET | /api/dashboard/resumo | Números do dashboard |

## 5.4 Regras de negócio

- **Status do débito** (PAGO, ATRASADO, PENDENTE) é calculado a cada consulta, nunca gravado.
- **Débito com itens** vale sempre a soma dos itens válidos; o valor digitado é ignorado.
- **Pagamento** registra data e autor; pagar duas vezes é recusado (código 409).
- **Campos desconhecidos são recusados:** não é possível criar débito já pago ou escolher o id.

- **Erros** voltam sempre como mensagem em português: 400 validação · 401 sem login · 404 não encontrado · 409 conflito · 429 excesso de tentativas.

### 5.5 Segurança

- Senhas com hash scrypt e sal aleatório; comparação em tempo constante.
- Token JWT com validade de 8 horas; segredo configurável por variável de ambiente.
- Bloqueio após 5 tentativas de login erradas em 15 minutos (por IP e usuário).
- Tempo de resposta igual para usuário existente e inexistente (não revela usuários válidos).
- Cabeçalhos de proteção: nosniff, X-Frame-Options DENY, Referrer-Policy e no-store na API.

## 6. Integração frontend ↔ backend

O frontend ganhou dois modos, escolhidos automaticamente pelo arquivo js/config.js:

| Como é aberto | Modo | Onde ficam os dados |
|---|---|---|
| http://localhost:8000 (pelo backend) | API | Banco SQLite no servidor |
| Duplo clique no index.html | Demonstração | Navegador (localStorage) |

O novo arquivo js/store-api.js tem exatamente as mesmas funções do js/store.js, mas conversa com o servidor. Por isso nenhuma tela precisou ser reescrita. As regras de validação são as mesmas nos dois modos.

## 7. Testes e verificação

### 7.1 Testes automatizados (16/16 passando)

- Login correto, senha errada, bloqueio por excesso de tentativas e rotas sem token.
- Senha nunca armazenada em texto puro.
- Cadastro, edição e exclusão de cliente; CPF inválido; CPF duplicado; campo extra recusado.
- Exclusão de cliente remove débitos e itens.
- Débito com carrinho usa a soma apenas dos itens válidos; valores negativos e datas invertidas recusados.
- Pagamento único (segunda tentativa recusada); cálculo do fuso de São Paulo.
- Dashboard, relatório de regiões e ranking de produtos conferidos contra os dados.

### 7.2 Testes no navegador

- Modo API: login, todas as páginas, cadastro de débito com carrinho pela tela e casos de erro — sem erros de JavaScript.
- Modo demonstração: base nova, base antiga com dados do usuário (preservados e migrados) e simulação de armazenamento cheio.
- Os dois modos exibem exatamente os mesmos números no dashboard e no perfil do cliente.

## 8. Inventário de arquivos

### 8.1 Backend (pasta backend/ — novo)

| Arquivo | Conteúdo |
|---|---|
| app/main.py | Aplicação, tratamento de erros, serve o frontend |

| Arquivo | Conteúdo |
|---|---|
| app/config.py | Configuração por variáveis de ambiente |
| app/db.py | Banco SQLite, esquema, datas e centavos |
| app/security.py | Senhas, tokens e limite de tentativas |
| app/validacao.py | CPF/CNPJ, telefone e e-mail |
| app/schemas.py | Validação dos dados recebidos |
| app/repo.py | Conversão do banco para o formato das telas |
| app/seed.py | Criação de usuários e dados de exemplo |
| app/rotas/*.py | Rotas: auth, clientes, debitos, relatorios |
| tests/ | 16 testes automatizados |
| README.md · requirements*.txt · .env.example · .gitignore | Documentação, dependências e configuração |

### 8.2 Frontend (pasta elogest-recebe-facil/)

| Arquivo | Mudança |
|---|---|
| js/config.js | Novo — escolhe o modo (API ou demonstração) |
| js/store-api.js | Novo — implementação via servidor |
| js/store.js | Reescrito — bugs 1 a 5, validação, centavos |
| js/format.js | Data local, CPF/CNPJ com dígito, escape |
| js/auth.js | Sessão com token |
| js/ui.js | Mensagens de erro do servidor; rodapé |
| js/registro-debito.js | Carrinho corrigido; débito numa operação só |
| js/cliente-detalhes.js | Ranking do mês e filtro via store |
| js/regioes.js, regioes.html | Ranking mostra valor em R$; título “Produtos que mais faturam” |
| js/index.js, clientes.js, cadastro-cliente.js, editar-cliente.js, cliente-validacao.js, contas-receber.js | Mensagens de erro mais claras |
| 9 páginas .html | Inclusão de config.js e store-api.js |
| README.md | Documentação dos dois modos |

## 9. Como usar e próximos passos

### 9.1 Iniciar o sistema

Na pasta backend, execute:

```
.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000
```

Depois abra http://localhost:8000 e entre com admin / 123456. A documentação interativa da API fica em http://localhost:8000/docs.

### 9.2 Antes de colocar em produção

1. Trocar ou remover o usuário admin/123456.
2. Definir a variável ELOGEST_JWT_SECRET com um valor longo e aleatório.
3. Publicar atrás de HTTPS (Nginx ou Caddy).
4. Agendar backup do arquivo backend/data/elogest.db.
5. Com vários servidores simultâneos, mover o limite de tentativas de login para Redis.

## 9.3 Pendências conhecidas

- Aviso de sucesso desaparece quando a página muda logo em seguida.
- Não há tela para editar ou excluir débitos, nem para pagamento parcial.
- “Ticket médio” mantém duas fórmulas diferentes (documentadas no código).
- Link do WhatsApp não trata telefones com 12 dígitos.
- Modo demonstração continua com senha em texto no navegador (aceitável por ser só demonstração).